Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н. Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н. Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления» (ИУ)

КАФЕДРА «Информационная безопасность» (ИУ8)

ЛАБОРАТОРНАЯ РАБОТА № 6

Изучение алгоритма обратного распространения ошибки (Back Propagation)

Вариант 2

Студент ИУ8-13М (Группа) Бочкарев Михаил Александрович (И. О. Фамилия) _____ (Подпись, дата)

Преподаватель Басараб Михаил Алексеевич (И. О. Фамилия) _____ (Подпись, дата)

Оценка: _____

Москва, 2026 г.

СОДЕРЖАНИЕ

1 Цель работы и постановка задачи 3
2 Алгоритм обратного распространения ошибки 4
2.1 Прямое распространение 4
2.2 Обратное распространение и коррекция весов 4
3 Численный эксперимент 6
3.1 Первый шаг обучения 6
3.2 Выходы сети и ошибка на каждой эпохе 7
3.3 Веса нейронов на каждой эпохе 8
3.4 Результат и проверка 9
4 Программная реализация 10
5 Выводы 11
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 12
ПРИЛОЖЕНИЕ А Исходный код программы 13
ПРИЛОЖЕНИЕ Б Построение графика ошибки 14

1 Цель работы и постановка задачи

Цель работы — исследовать функционирование многослойной нейронной сети прямого распространения и реализовать её обучение методом обратного распространения ошибки.

В соответствии с вариантом 2 таблицы 6.2 методических указаний [1] используется архитектура 1–1–3: один входной сигнал, один скрытый нейрон и три выходных нейрона. Обучающая выборка содержит одну пару входного и целевого векторов:

$$\mathbf{x} = (1, -1), \qquad \mathbf{t} = (0{,}2, -0{,}3, 0{,}1).$$

Первый элемент входного вектора $x_0 = 1$ является постоянным входом смещения, а $x_1 = -1$ — входным сигналом. В таблице вариантов приведён столбец $10\mathbf{t}$, поэтому значения $(2, -3, 1)$ разделены на 10.

Необходимо реализовать прямой проход, вычисление ошибок обоих слоёв и коррекцию весов; получить выходы сети, суммарную ошибку и наборы весов на каждой эпохе. Приняты скорость обучения $\eta = 1$ и порог $\varepsilon = 10^{-3}$. Начальные веса выбраны небольшими ненулевыми числами, что допускается теоретической частью методических указаний.

Обозначим веса скрытого нейрона через a_0, a_1, смещение выходного нейрона m через b_m, а вес его связи со скрытым нейроном через v_m. Всего сеть имеет восемь обучаемых параметров:

$$(a_0, a_1) = (0{,}1, -0{,}2),$$

$$(b_1, v_1) = (0{,}05, 0{,}2), \quad (b_2, v_2) = (-0{,}1, -0{,}15),$$

$$(b_3, v_3) = (0{,}15, 0{,}1).$$

В каждой паре первый параметр соответствует смещению, второй — входящей связи. Эти начальные значения фиксированы для воспроизводимости эксперимента.

2 Алгоритм обратного распространения ошибки

2.1 Прямое распространение

Алгоритм соответствует лекционным материалам [2, 3]. Для обоих слоёв используется заданная методическими указаниями функция активации:

$$f(s) = \frac{1 - \exp(-s)}{1 + \exp(-s)} = \tanh\frac{s}{2}, \qquad f'(s) = \frac{1}{2}(1 - f^2(s)). \tag{1}$$

Функция принимает значения в интервале $(-1, 1)$ и допускает отрицательные целевые значения. Её нелинейность позволяет строить нелинейные отображения, а дифференцируемость необходима для вычисления градиента.

Скрытый нейрон формирует сигнал

$$net_h = a_0 + a_1 x_1, \qquad h = f(net_h). \tag{2}$$

Затем каждый выходной нейрон вычисляет

$$net_m = b_m + v_m h, \qquad y_m = f(net_m), \qquad m = 1, 2, 3. \tag{3}$$

Для остановки используется ошибка из методических указаний:

$$E(k) = \sqrt{\sum_{m=1}^{3} (t_m - y_m(k))^2} \le \varepsilon. \tag{4}$$

Это евклидова норма вектора ошибок; деление на число выходов в данной формуле отсутствует.

2.2 Обратное распространение и коррекция весов

Градиентные обновления соответствуют минимизации функции потерь

$$L = \frac{1}{2}\sum_{m=1}^{3} (t_m - y_m)^2 = \frac{1}{2}E^2. \tag{5}$$

Для выходных нейронов целевые значения известны, поэтому сигналы ошибки равны

$$\delta_m = \frac{1}{2}(1 - y_m^2)(t_m - y_m). \tag{6}$$

Целевое значение скрытого нейрона не задаётся. По правилу дифференцирования сложной функции его сигнал ошибки выражается через выходной слой:

$$\delta_h = \frac{1}{2}(1 - h^2) \sum_{m=1}^{3} v_m \delta_m. \tag{7}$$

Таким образом, ошибки выходных нейронов распространяются назад через веса связей. Все сигналы ошибки рассчитываются по весам текущей эпохи, до их изменения.

Обновления параметров имеют вид

$$\begin{aligned} a_0(k+1) &= a_0(k) + \eta \delta_h, \\ a_1(k+1) &= a_1(k) + \eta x_1 \delta_h, \\ b_m(k+1) &= b_m(k) + \eta \delta_m, \\ v_m(k+1) &= v_m(k) + \eta h \delta_m. \end{aligned} \tag{8}$$

Знак «плюс» согласован с определением ошибки как $t_m - y_m$. Для смещений вход равен единице. После обновления выполняется следующий прямой проход и проверяется условие остановки.

При нулевой инициализации всех весов $h = 0$, $v_m = 0$, поэтому $\delta_h = 0$ и $\Delta v_m = 0$. В таком случае изменяются только выходные смещения. В настоящем эксперименте ненулевая инициализация позволяет продемонстрировать обучение обоих слоёв.

3 Численный эксперимент

3.1 Первый шаг обучения

В таблицах номер k означает количество выполненных обновлений: $k = 0$ — исходное состояние, а $k = 1$ — состояние после первой коррекции. Одна эпоха содержит предъявление единственной обучающей пары.

При начальных весах получаем

$$net_h = 0{,}1 + (-0{,}2)(-1) = 0{,}3, \qquad h = \tanh(0{,}15).$$

$$h \approx 0.148885.$$

$$\mathbf{net}_{out} \approx (0.079777,\ -0.122333,\ 0.164889).$$

$$\mathbf{y}(0) \approx (0.039867,\ -0.061090,\ 0.082258).$$

$$E(0) \approx 0.288158.$$

$$\boldsymbol{\delta}_{out} \approx (0.079939,\ -0.119009,\ 0.008811).$$

$$\delta_h \approx 0.016975.$$

Поскольку $\delta_h \neq 0$, корректируются и входной вес скрытого нейрона, и его смещение. После первого обновления:

$$(a_0, a_1) \approx (0.116975,\ -0.216975).$$

$$(b_1, v_1) \approx (0.129939,\ 0.211902).$$

$$(b_2, v_2) \approx (-0.219009,\ -0.167719).$$

$$(b_3, v_3) \approx (0.158811,\ 0.101312).$$

$$\mathbf{y}(1) \approx (0.082311,\ -0.122756,\ 0.087561).$$

Ошибка после первого обновления равна $E(1) \approx 0.213121$.

3.2 Выходы сети и ошибка на каждой эпохе

Целевой вектор на каждой строке таблицы один и тот же: $\mathbf{t} = (0{,}2, -0{,}3, 0{,}1)$. Значения в таблицах округлены до шести знаков после запятой; критерий остановки проверяется по неокруглённым числам.

Таблица 1 – Скрытый сигнал, выход сети и суммарная ошибка

k	h	y_1	y_2	y_3	$E(k)$
0	0.148885	0.039867	-0.061090	0.082258	0.288158
1	0.165441	0.082311	-0.122756	0.087561	0.213121
2	0.178489	0.113475	-0.167802	0.091368	0.158231
3	0.188594	0.136303	-0.200760	0.094074	0.118071
4	0.196342	0.153040	-0.225047	0.095979	0.088540
5	0.202254	0.165335	-0.243095	0.097308	0.066686
6	0.206751	0.174387	-0.256611	0.098228	0.050416
7	0.210170	0.181063	-0.266803	0.098858	0.038236
8	0.212766	0.185994	-0.274530	0.099285	0.029076
9	0.214738	0.189639	-0.280415	0.099570	0.022161
10	0.216237	0.192337	-0.284913	0.099758	0.016923
11	0.217376	0.194333	-0.288362	0.099879	0.012945
12	0.218242	0.195811	-0.291012	0.099955	0.009916
13	0.218901	0.196905	-0.293053	0.100001	0.007605
14	0.219402	0.197715	-0.294626	0.100027	0.005840
15	0.219784	0.198314	-0.295840	0.100040	0.004489
16	0.220075	0.198757	-0.296778	0.100044	0.003454
17	0.220296	0.199084	-0.297504	0.100045	0.002659
18	0.220465	0.199326	-0.298065	0.100042	0.002049
19	0.220594	0.199505	-0.298500	0.100038	0.001580
20	0.220692	0.199636	-0.298837	0.100033	0.001219
21	0.220767	0.199733	-0.299098	0.100029	0.000941

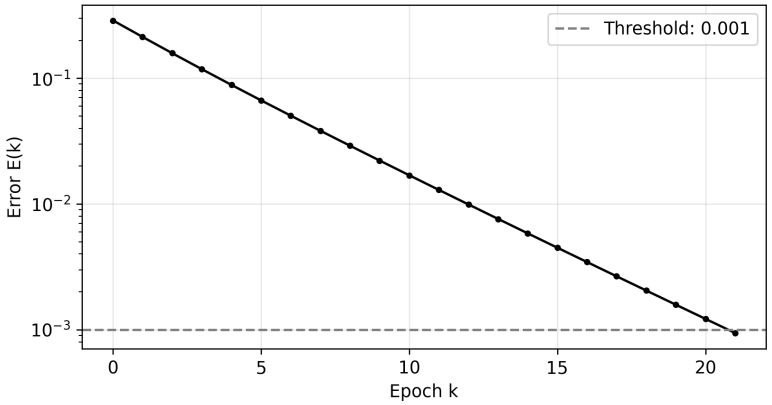


Рисунок 1 – Зависимость ошибки от эпохи обучения; шкала ошибки логарифмическая

3.3 Веса нейронов на каждой эпохе

Веса разделены на две таблицы для удобства чтения. Совокупность строк с одинаковым k содержит все восемь параметров сети.

Таблица 2 – Веса скрытого и первого выходного нейронов

k	a_0	a_1	b_1	v_1
0	0.100000	-0.200000	0.050000	0.200000
1	0.116975	-0.216975	0.129939	0.211902
2	0.130422	-0.230422	0.188385	0.221571
3	0.140878	-0.240878	0.231090	0.229193
4	0.148925	-0.248925	0.262346	0.235088
5	0.155081	-0.255081	0.285276	0.239590
6	0.159775	-0.259775	0.302135	0.243000
7	0.163349	-0.263349	0.314552	0.245567
8	0.166067	-0.266067	0.323710	0.247492
9	0.168134	-0.268134	0.330471	0.248931
10	0.169705	-0.269705	0.335465	0.250003
11	0.170900	-0.270900	0.339155	0.250801
12	0.171809	-0.271809	0.341881	0.251393
13	0.172501	-0.272501	0.343895	0.251833
14	0.173028	-0.273028	0.345383	0.252159
15	0.173429	-0.273429	0.346481	0.252399
16	0.173734	-0.273734	0.347290	0.252577
17	0.173967	-0.273967	0.347887	0.252709
18	0.174145	-0.274145	0.348327	0.252806
19	0.174280	-0.274280	0.348650	0.252877
20	0.174384	-0.274384	0.348888	0.252929
21	0.174462	-0.274462	0.349063	0.252968

Таблица 3 – Веса второго и третьего выходных нейронов

k	b_2	v_2	b_3	v_3
0	-0.100000	-0.150000	0.150000	0.100000
1	-0.219009	-0.167719	0.158811	0.101312
2	-0.306296	-0.182159	0.164983	0.102333
3	-0.370533	-0.193625	0.169262	0.103097
4	-0.418153	-0.202606	0.172199	0.103651
5	-0.453731	-0.209591	0.174191	0.104042
6	-0.480502	-0.215006	0.175525	0.104311
7	-0.500768	-0.219196	0.176402	0.104493
8	-0.516185	-0.222436	0.176968	0.104612
9	-0.527961	-0.224942	0.177322	0.104687
10	-0.536983	-0.226879	0.177535	0.104733
11	-0.543914	-0.228378	0.177654	0.104759
12	-0.549249	-0.229538	0.177714	0.104772
13	-0.553362	-0.230435	0.177737	0.104777
14	-0.556538	-0.231130	0.177736	0.104776
15	-0.558992	-0.231669	0.177723	0.104774
16	-0.560890	-0.232086	0.177703	0.104769
17	-0.562359	-0.232409	0.177681	0.104764
18	-0.563496	-0.232660	0.177659	0.104760
19	-0.564378	-0.232854	0.177639	0.104755
20	-0.565061	-0.233005	0.177620	0.104751
21	-0.565591	-0.233122	0.177604	0.104747

3.4 Результат и проверка

Условие остановки впервые выполнено при $K = 21$:

$$\mathbf{y}(K) \approx (0.19973342,\ -0.29909753,\ 0.10002854).$$

$$E(K) \approx 0.00094145 < 0{,}001.$$

На предыдущей эпохе $E(K-1) \approx 0.00121931 > 0{,}001$. Таким образом, выполнено 21 обновление весов; таблицы содержат 22 состояния с учётом исходного. Ошибка монотонно уменьшается в данном эксперименте. Все восемь весов изменились относительно начальных значений. При $x_1 = -1$ приращения a_0 и a_1 противоположны, поэтому сумма $a_0 + a_1 = -0{,}1$ сохраняется. Это дополнительно подтверждает согласованность формул обновления.

4 Программная реализация

Программа написана на Python 3 и использует только стандартные модули `math` и `csv`. Она выводит вход, цель, скрытый сигнал, выход, ошибку и веса каждого нейрона. Файл `training.csv` содержит также взвешенные суммы и сигналы ошибки всех нейронов на каждой эпохе. Защитный предел составляет 10000 обновлений. Для запуска используется команда `python lab6.py`.

Вспомогательная программа построения графика приведена в приложении Б, листинг Б.2. Полный исходный код программы приведён в приложении А, листинг А.1. В программе реализованы прямое распространение сигналов, вычисление ошибок выходного и скрытого слоёв, обновление всех восьми весов и проверка условия остановки.

5 Выводы

Реализована многослойная нейронная сеть архитектуры 1–1–3 с заданной функцией активации $f(s) = \tanh(s/2)$. Метод обратного распространения ошибки обеспечивает вычисление поправок выходного слоя по разности цели и выхода, а скрытого слоя — через взвешенную сумму выходных сигналов ошибки.

При фиксированной ненулевой инициализации и $\eta = 1$ заданная точность $\varepsilon = 10^{-3}$ достигнута за 21 эпоху. Итоговая ошибка составляет приблизительно 0,00094145. В отличие от нулевой инициализации, в проведённом эксперименте изменяются веса обоих слоёв.

Эксперимент подтверждает восстановление целевого вектора для заданного входа. Поскольку обучающая выборка содержит единственную пару, полученный результат не позволяет оценить обобщающую способность сети или качество работы на других входных данных.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

а) Методические указания. Лабораторная работа № 6 «Изучение алгоритма обратного распространения ошибки (метод Back Propagation)». Предоставленные страницы 33–36.

б) Многослойные НС прямого распространения. Учебная презентация, файл `L4a_Многослойные.pdf`. 29 слайдов.

в) Басараб М. А., Коннова Н. С. Метод обратного распространения ошибки (Back Propagation). Пример: архитектура 1–1–2 (одна эпоха обучения). МГТУ им. Н. Э. Баумана, кафедра ИУ8. 11 слайдов.

ПРИЛОЖЕНИЕ А

Исходный код программы

Листинг А.1 – Программа обучения сети архитектуры 1–1–3

```
from math import tanh, sqrt
from pathlib import Path
import csv

ROOT = Path(__file__).resolve().parent

x = [1.0, -1.0]
t = [0.2, -0.3, 0.1]
w_hidden = [0.1, -0.2]
w_output = [[0.05, 0.2], [-0.1, -0.15], [0.15, 0.1]]
eta = 1.0
epsilon = 0.001
max_epochs = 10000
history = []

def activation(net):
    return tanh(net / 2.0)

def derivative(output):
    return 0.5 * (1.0 - output ** 2)

for epoch in range(max_epochs + 1):
    net_h = sum(w * value for w, value in zip(w_hidden, x))
    hidden = activation(net_h)
    net_y = [w[0] + w[1] * hidden for w in w_output]
    y = [activation(net) for net in net_y]
    error = sqrt(sum((target - out) ** 2 for target, out in zip(t, y)))
    delta_y = [derivative(out) * (target - out) for target, out in zip(t, y)]
    delta_h = derivative(hidden) * sum(w[1] * d for w, d in zip(w_output, delta_y))
    weights = w_hidden + [value for pair in w_output for value in pair]
    history.append([epoch, *x, *t, net_h, hidden, *net_y, *y, error, *weights, delta_h, *delta_y])
    print(f"k={epoch}; x={x}; t={t}; h={hidden:.8f}; y={y}; E={error:.8f}")
    print(f"w_hidden={w_hidden}; w_y1={w_output[0]}; w_y2={w_output[1]}; w_y3={w_output[2]}")
    if error <= epsilon:
        break
    if epoch == max_epochs:
        raise RuntimeError("Target accuracy was not reached")
    for i in range(len(w_hidden)):
        w_hidden[i] += eta * x[i] * delta_h
    for m in range(len(w_output)):
        w_output[m][0] += eta * delta_y[m]
        w_output[m][1] += eta * hidden * delta_y[m]

header = ["k", "x0", "x1", "t1", "t2", "t3", "net_h", "h"]
header += ["net1", "net2", "net3", "y1", "y2", "y3", "E"]
header += ["a0", "a1", "b1", "v1", "b2", "v2", "b3", "v3"]
header += ["delta_h", "delta1", "delta2", "delta3"]
with (ROOT / "training.csv").open("w", newline="", encoding="utf-8") as file:
    writer = csv.writer(file)
    writer.writerow(header)
    writer.writerows(history)
```

ПРИЛОЖЕНИЕ Б

Построение графика ошибки

Листинг Б.2 – Построение графика по сохранённым результатам

```
from pathlib import Path
import csv
import matplotlib.pyplot as plt

ROOT = Path(__file__).resolve().parent
with (ROOT / "training.csv").open(encoding="utf-8") as file:
    rows = list(csv.DictReader(file))
epochs = [int(row["k"]) for row in rows]
errors = [float(row["E"]) for row in rows]
plt.rcParams.update({"font.family": "DejaVu Sans", "font.size": 11})
fig, ax = plt.subplots(figsize=(7.0, 3.8))
ax.semilogy(epochs, errors, "o-", color="black", markersize=3)
ax.axhline(0.001, color="gray", linestyle="--", label="Threshold: 0.001")
ax.set_xlabel("Epoch k")
ax.set_ylabel("Error E(k)")
ax.grid(True, alpha=0.3)
ax.legend()
fig.tight_layout()
fig.savefig(ROOT / "error.png", dpi=240)
plt.close(fig)
```